Slip 1

Q1. Create a dictionary containing mobile names and their prices. Create DataFrame from this dictionary. Display the contents of DataFrame.

```
import pandas as pd

mobiles = {
    "Mobile": ["Samsung", "Apple", "Redmi", "OnePlus", "Vivo"],
    "Price": [25000, 80000, 12000, 35000, 18000]
}
df = pd.DataFrame(mobiles)
print(df)
```

Q2. Create a package **vehicle** with sub-package **types** containing modules **cars.py** and **bikes.py**. Each module has `available_models()`. Display all models.

```
# File: vehicle/__init__.py  → (empty)
# File: vehicle/types/__init__.py  → (empty)

# File: vehicle/types/cars.py
def available_models():
    return ["Swift", "Innova", "Creta", "Nexon"]

# File: vehicle/types/bikes.py
def available_models():
    return ["Splendor", "Pulsar", "Activa", "Duke"]

# File: main.py
from vehicle.types import cars, bikes

print("Car Models:", cars.available_models())
print("Bike Models:", bikes.available_models())
```

Q2 (OR). List all files in current directory. Display total count of .py files and content of any one .py file.

```
import os

files = os.listdir(".")
print("All files:", files)

py_files = [f for f in files if f.endswith(".py")]
print("Total .py files:", len(py_files))

if py_files:
    with open(py_files[0], "r") as f:
        print(f"\nContents of {py_files[0]}:")
        print(f.read())
```

Slip 2

Q1. Accept a string. Count total number of vowels, consonants and special symbols.

```
s = input("Enter string: ")
vowels = consonants = special = 0
for ch in s:
    if ch.lower() in "aeiou":
        vowels += 1
    elif ch.isalpha():
        consonants += 1
    elif not ch.isspace():
        special += 1
print("Vowels:", vowels)
print("Consonants:", consonants)
print("Special Symbols:", special)
```

Q2. Create IPL DataFrame. a) First 2 rows b) Last 3 rows c) Check null values d) Replace null with mean e) Most expensive player f) Total players per team.

```
import pandas as pd
import numpy as np

data = {
    "Player": ["Hardik Pandya","K L Rahul","Andre Russel","Jasprit Bumrah","Virat Kohli","Rohit
Sharma"],
    "Team": ["Mumbai Indians","Kings Eleven","Kolkata Knight Riders","Mumbai Indians","RCB","Mumbai
Indians"],
    "Category": ["Batsman","Batsman","Batsman","Bowler","Batsman","Batsman"],
    "BidPrice": [13, 12, np.nan, 10, 17, np.nan],
    "Runs": [1000, 2400, 900, 200, 3600, 3700]
}
df = pd.DataFrame(data)

print("First 2 rows:\n", df.head(2))
print("\nLast 3 rows:\n", df.tail(3))
print("\nNull values:\n", df.isnull().sum())
df["BidPrice"].fillna(df["BidPrice"].mean(), inplace=True)
print("\nAfter replacing null:\n", df)
print("\nMost expensive:", df.loc[df["BidPrice"].idxmax(), "Player"])
print("\nPlayers per team:\n", df["Team"].value_counts())
```

Q2 (OR). User-defined exception for marks validation (0–100). Display grade.

```
class InvalidMarksError(Exception):
    pass

try:
    marks = int(input("Enter marks: "))
    if not isinstance(marks, int):
        raise InvalidMarksError("Invalid input.")
    if marks < 0 or marks > 100:
        raise InvalidMarksError("Marks out of range (0-100).")
    if marks >= 90: grade = "A+"
    elif marks >= 75: grade = "A"
    elif marks >= 60: grade = "B"
    elif marks >= 40: grade = "C"
    else: grade = "F"
    print("Grade:", grade)
except InvalidMarksError as e:
    print("Error:", e)
except ValueError:
    print("Error: Please enter a valid integer.")
```

Slip 3

Q1. Accept m and n. Divide m by n. Handle ZeroDivisionError.

```
try:
    m = float(input("Enter m: "))
    n = float(input("Enter n: "))
    result = m / n
    print("Result:", result)
except ZeroDivisionError:
    print("Error: Cannot divide by zero.")
```

Q2. Create module **variables.py** with factorial function, power dictionary, list of vowels. Import and use it.

```
# File: variables.py
import math
def factorial(n):
    return math.factorial(n)

power = {1:1, 2:4, 3:9, 4:16, 5:25}
vowels = ['a', 'e', 'i', 'o', 'u']

# File: main.py
import variables

n = int(input("Enter n: "))
print("Factorial of", n, "=", variables.factorial(n))
print("Power of", n, "=", variables.power.get(n, n**2))
print("Second vowel:", variables.vowels[1])
```

Q2 (OR). Accept filename. If file exists display contents, count characters, words, lines.

```
import os

fname = input("Enter filename: ")
if os.path.exists(fname):
    with open(fname, "r") as f:
        content = f.read()
    print("Contents:\n", content)
    print("Characters:", len(content))
    print("Words:", len(content.split()))
    print("Lines:", content.count('\n') + 1)
else:
    print("File does not exist.")
```

Slip 4

Q1. Create a list. Shuffle elements using random.shuffle(). Display list.

```
import random

lst = [10, 20, 30, 40, 50, 60, 70, 80]
print("Original:", lst)
random.shuffle(lst)
print("Shuffled:", lst)
```

Q2. Create stationery DataFrame. a) Find Pencil rows b) Change Eraser count to 25 c) List Company and Price d) Pencil and Pen rows e) Rename Count to Quantity.

```
import pandas as pd

data = {
    "Company": ["Apsara","Natraj","Cello","Parkar","Apsara"],
    "Count": [15, 20, 25, 35, 20],
    "Price": [250, 200, 600, 900, 300]
}
index = ["Pencil","Pencil","Pen","Pen","Eraser"]
df = pd.DataFrame(data, index=index)

print("a) Pencil rows:\n", df.loc["Pencil"])
df.loc["Eraser","Count"] = 25
print("\nb) After changing Eraser count:\n", df)
print("\nc) Company and Price:\n", df[["Company","Price"]])
print("\nd) Pencil and Pen:\n", df.loc[["Pencil","Pen"]])
df.rename(columns={"Count":"Quantity"}, inplace=True)
print("\ne) After renaming:\n", df)
```

Q2 (OR). File operations: create file with 5 friend names, display, append 5 more, display again.

```
fname = "friends.txt"
friends1 = ["Amit Patil", "Ravi Sharma", "Priya Joshi", "Sneha More", "Raj Kumar"]
friends2 = ["Neha Singh", "Suresh Desai", "Kavita Nair", "Arjun Mehta", "Pooja Verma"]

with open(fname, "w") as f:
    for name in friends1:
        f.write(name + "\n")

with open(fname, "r") as f:
    print("Original:\n", f.read())

with open(fname, "a") as f:
    for name in friends2:
        f.write(name + "\n")

with open(fname, "r") as f:
    print("Modified:\n", f.read())
```

Slip 5

Q1. Accept a string, find all words that do not contain vowels.

```
import re

s = input("Enter string: ")
words = s.split()
no_vowel = [w for w in words if not re.search(r'[aeiouAEIOU]', w)]
print("Words without vowels:", no_vowel)
```

Q2. Create package **banking** with modules **account.py** and **loan.py**. Demonstrate both functions.

```
# File: banking/__init__.py  → (empty)

# File: banking/account.py
def open_account(name):
    print(f"Account successfully created for {name}.")

# File: banking/loan.py
def apply_loan(amount):
    print(f"Loan of Rs.{amount} applied successfully.")

# File: main.py
from banking import account, loan

name = input("Enter your name: ")
account.open_account(name)
amount = float(input("Enter loan amount: "))
loan.apply_loan(amount)
```

Q2 (OR). Create list of prime numbers between m and n using list comprehension.

```
def is_prime(n):
    if n < 2: return False
    for i in range(2, int(n**0.5)+1):
        if n % i == 0: return False
    return True

m = int(input("Enter m: "))
n = int(input("Enter n: "))
primes = [x for x in range(m, n+1) if is_prime(x)]
print("Primes between", m, "and", n, ":", primes)
```

Slip 6

Q1. Accept two numbers. Raise TypeError if inputs are not numerical.

```
try:
    a = input("Enter first number: ")
    b = input("Enter second number: ")
    if not a.replace('.','',1).lstrip('-').isdigit():
        raise TypeError("First input is not a number.")
    if not b.replace('.','',1).lstrip('-').isdigit():
        raise TypeError("Second input is not a number.")
    print("Sum:", float(a) + float(b))
except TypeError as e:
    print("TypeError:", e)
```

Q2. Create stationery DataFrame. a) Pencil rows b) Change Eraser count to 25 c) Company and Price d) Pencil and Pen rows e) Delete Count column.

```
import pandas as pd

data = {
    "Company": ["Apsara","Natraj","Cello","Parkar","Apsara"],
    "Count": [15, 20, 25, 35, 20],
    "Price": [250, 200, 600, 900, 300]
}
index = ["Pencil","Pencil","Pen","Pen","Eraser"]
df = pd.DataFrame(data, index=index)

print("a) Pencil rows:\n", df.loc["Pencil"])
df.loc["Eraser","Count"] = 25
print("\nb) After change:\n", df)
print("\nc) Company & Price:\n", df[["Company","Price"]])
print("\nd) Pencil & Pen:\n", df.loc[["Pencil","Pen"]])
df.drop(columns=["Count"], inplace=True)
print("\ne) After deleting Count:\n", df)
```

Q2 (OR). Flask Personal Portfolio Page.

```
# File: app.py
from flask import Flask
app = Flask(__name__)

@app.route('/')
def portfolio():
    return '''
<html><body>
<h1 style="text-align:center">Sumit Patil</h1>
<h3>Qualification:</h3>
<ul>
  <li>S.Y.B.Sc. Computer Science</li>
  <li>SPPU, Pune</li>
</ul>
<marquee>Skills: Python | Flask | HTML | CSS | SQL</marquee>
</body></html>'''

if __name__ == '__main__':
    app.run(debug=True)
```

Slip 7

Q1. Accept string. Extract words starting with capital letter. Remove all special characters. Display modified string.

```
import re

s = input("Enter string: ")
capital_words = re.findall(r'\b[A-Z][a-z]*', s)
print("Words starting with capital:", capital_words)
modified = re.sub(r'[^a-zA-Z0-9 ]', '', s)
print("After removing special chars:", modified)
```

Q2. Flask app showing Library Book List with table and quote.

```
# File: app.py
from flask import Flask
app = Flask(__name__)

@app.route('/')
def library():
    return '''
<html><body>
<h1>City Public Library</h1>
<table border="1">
  <tr><th>Title</th><th>Author</th></tr>
  <tr><td>Python Crash Course</td><td>Eric Matthes</td></tr>
  <tr><td>Clean Code</td><td>Robert Martin</td></tr>
  <tr><td>The Alchemist</td><td>Paulo Coelho</td></tr>
</table>
<p><b style="color:blue">"A reader lives a thousand lives before he dies."</b></p>
</body></html>'''

if __name__ == '__main__':
    app.run(debug=True)
```

Q2 (OR). Accept filename. If exists, copy to new file, display new file with count of lines, words, chars.

```
import os

src = input("Enter source filename: ")
if os.path.exists(src):
    dst = "copy_" + src
    with open(src, "r") as f:
        content = f.read()
    with open(dst, "w") as f:
        f.write(content)
    print("Copied to:", dst)
    print("Contents:\n", content)
    print("Lines:", content.count('\n')+1)
    print("Words:", len(content.split()))
    print("Characters:", len(content))
else:
    print("File does not exist.")
```

Slip 8

Q1. Accept radius. Calculate area of circle using function (π = 3.14).

```
def area_of_circle(r):
    return 3.14 * r * r

r = float(input("Enter radius: "))
print("Area of circle:", area_of_circle(r))
```

Q2. Create list, string and tuple. Use random module to display unique multiple random elements from each.

```
import random

lst = [10, 20, 30, 40, 50, 60]
s = "Hello World Python"
tup = (1, 2, 3, 4, 5, 6, 7, 8)

print("Random from list:", random.sample(lst, 3))
print("Random from string:", random.sample(s, 4))
print("Random from tuple:", random.sample(tup, 3))
```

Q2 (OR). Flask app showing College info with name, courses, and image.

```
# File: app.py
from flask import Flask
app = Flask(__name__)

@app.route('/')
def college():
    return '''
<html><body>
<h1 style="text-align:center">SPPU College of Science</h1>
<h3>Courses Available:</h3>
<ul>
  <li>B.Sc. Computer Science</li>
  <li>B.Sc. Mathematics</li>
  <li>B.Sc. Physics</li>
</ul>
<img src="https://via.placeholder.com/400x200?text=College+Image" alt="College">
</body></html>'''

if __name__ == '__main__':
    app.run(debug=True)
```

Slip 9

Q1. Accept n. Create a generator that yields even numbers up to n.

```
def even_gen(n):
    for i in range(2, n+1, 2):
        yield i

n = int(input("Enter n: "))
for val in even_gen(n):
    print(val, end=" ")
```

Q2. Create Student1 and Student2 DataFrames. Join with matching records from both sides (inner join on ID).

```
import pandas as pd

s1 = pd.DataFrame({
    "ID": ["S1","S2","S3","S4","S5"],
    "Name": ["Anita","Sakshi","Om","Raj","Kanika"],
    "Age": [18, 23, 20, 21, 20]
})
s2 = pd.DataFrame({
    "ID": ["S2","S4","S5","S6","S7"],
    "City": ["Pune","Nashik","Mumbai","Delhi","Nanded"],
    "Marks": [97, 78, 86, 66, 54]
})
print("Student1:\n", s1)
print("\nStudent2:\n", s2)
s3 = pd.merge(s1, s2, on="ID", how="inner")
print("\nJoined DataFrame:\n", s3)
```

Q2 (OR). Accept number. a) Check Armstrong b) Sum of digits till single digit.

```
def is_armstrong(n):
    digits = str(n)
    power = len(digits)
    return n == sum(int(d)**power for d in digits)

def digit_sum(n):
    while n >= 10:
        n = sum(int(d) for d in str(n))
    return n

n = int(input("Enter number: "))
print("Armstrong:", is_armstrong(n))
print("Sum till single digit:", digit_sum(n))
```

Slip 10

Q1. Create DataFrame from list of 10 integers. Display contents and calculate sum, mean, median, mode, std dev.

```
import pandas as pd

data = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
df = pd.DataFrame(data, columns=["Values"])
print(df)
print("Sum:", df["Values"].sum())
print("Mean:", df["Values"].mean())
print("Median:", df["Values"].median())
print("Mode:", df["Values"].mode()[0])
print("Std Dev:", df["Values"].std())
```

Q2. Create package **finance_tools** with **interest.py** and **currency.py**. Demonstrate both functions.

```
# File: finance_tools/__init__.py  → (empty)

# File: finance_tools/interest.py
def cal_interest(amount, year):
    rate = 5
    si = (amount * rate * year) / 100
    return si

# File: finance_tools/currency.py
def convert(amount):
    return amount * 0.012

# File: main.py
from finance_tools import interest, currency

amount = float(input("Enter amount: "))
year = int(input("Enter years: "))
print("Simple Interest:", interest.cal_interest(amount, year))
print("Amount in USD:", currency.convert(amount))
```

Q2 (OR). Create a class iterator that generates factorial series up to n.

```
class FactorialSeries:
    def __init__(self, n):
        self.n = n
        self.current = 1
        self.fact = 1

    def __iter__(self):
        return self

    def __next__(self):
        if self.current > self.n:
            raise StopIteration
        self.fact *= self.current
        self.current += 1
        return self.fact

n = int(input("Enter n: "))
for val in FactorialSeries(n):
    print(val, end=" ")
```

Slip 11

Q1. Create list of 10 integers. Display square of all elements using lambda with map().

```
lst = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
squares = list(map(lambda x: x**2, lst))
print("Squares:", squares)
```

Q2. Flask app to add and view tasks (using in-memory list instead of PostgreSQL for portability).

```
# File: app.py
from flask import Flask, request, redirect, url_for
app = Flask(__name__)
tasks = []

@app.route('/', methods=['GET','POST'])
def index():
    if request.method == 'POST':
        task = request.form.get('task_name')
        if task:
            tasks.append(task)
        return redirect(url_for('index'))
    task_list = "".join(f"<li>{t}</li>" for t in tasks)
    return f'''
<html><body>
<h2>Task Manager</h2>
<form method="POST">
  Task: <input name="task_name">
  <button type="submit">Add</button>
</form>
<ul>{task_list}</ul>
</body></html>'''

if __name__ == '__main__':
    app.run(debug=True)
```

Q2 (OR). Accept string. a) Split into words b) Find words without vowels c) Sort alphabetically d) Display count.

```
import re

s = input("Enter string: ")
words = s.split()
no_vowel = [w for w in words if not re.search(r'[aeiou]', w, re.IGNORECASE)]
no_vowel_sorted = sorted(no_vowel)
print("Words without vowels:", no_vowel_sorted)
print("Total count:", len(no_vowel_sorted))
```

Slip 12

Q1. Create data.csv with employee name, designation, salary. Create DataFrame from csv and display with statistical info.

```
import pandas as pd

# Create data.csv first
csv_content =
"Name,Designation,Salary\nAlice,Manager,75000\nBob,Developer,60000\nCarol,Designer,55000\nDave,Analyst,5
0000\n"
with open("data.csv", "w") as f:
    f.write(csv_content)

df = pd.read_csv("data.csv")
print(df)
print("\nStatistical Info:\n", df.describe())
```

Q2. a) Get local and UTC time b) Print using ctime() and gmtime() c) Accept datetime string d) Parse with strptime() and format with strftime() in two formats.

```
import time

local = time.localtime()
utc = time.gmtime()
print("Local time:", time.ctime())
print("UTC time:", time.strftime("%Y-%m-%d %H:%M:%S", utc))

dt_str = input("Enter datetime (DD-MM-YYYY HH:MM:SS): ")
t = time.strptime(dt_str, "%d-%m-%Y %H:%M:%S")
print("Format 1:", time.strftime("%A, %d %B %Y", t))
print("Format 2:", time.strftime("%d/%m/%Y %I:%M %p", t))
```

Q2 (OR). Create list, string and tuple. Use random module to display unique multiple random elements.

```
import random

lst = [10, 20, 30, 40, 50, 60]
s = "Hello World Python"
tup = (1, 2, 3, 4, 5, 6, 7, 8)

print("Random from list:", random.sample(lst, 3))
print("Random from string:", random.sample(s, 4))
print("Random from tuple:", random.sample(tup, 3))
```

Slip 13

Q1. Get current time. a) Convert seconds to readable string (ctime) b) Convert to struct_time in UTC (gmtime).

```
import time

t = time.time()
print("Current time (ctime):", time.ctime(t))
utc = time.gmtime(t)
print("UTC struct_time:", utc)
print("Formatted UTC:", time.strftime("%Y-%m-%d %H:%M:%S", utc))
```

Q2. Accept text filename. If exists, display line by line with ftell() position. Then seek(0) and display again.

```
import os

fname = input("Enter filename: ")
if os.path.exists(fname):
    with open(fname, "r") as f:
        print("Reading line by line:")
        for line in f:
            print(line, end="")
            print("  [Position:", f.tell(), "]")
        f.seek(0)
        print("\nAfter seek(0):\n", f.read())
else:
    print("File does not exist.")
```

Q2 (OR). Create Student1 and Student2 DataFrames. Join with matching records (inner join on ID).

```
import pandas as pd

s1 = pd.DataFrame({
    "ID": ["S1","S2","S3","S4","S5"],
    "Name": ["Anita","Sakshi","Om","Raj","Kanika"],
    "Age": [18, 23, 20, 21, 20]
})
s2 = pd.DataFrame({
    "ID": ["S2","S4","S5","S6","S7"],
    "City": ["Pune","Nashik","Mumbai","Delhi","Nanded"],
    "Marks": [97, 78, 86, 66, 54]
})
s3 = pd.merge(s1, s2, on="ID", how="inner")
print("Student1:\n", s1)
print("\nStudent2:\n", s2)
print("\nJoined:\n", s3)
```

Slip 14

Q1. Accept string and substring. Check if substring is present at beginning of the string using RE module.

```
import re

s = input("Enter string: ")
sub = input("Enter substring: ")
if re.match(re.escape(sub), s):
    print(f"'{sub}' is present at the beginning of the string.")
else:
    print(f"'{sub}' is NOT at the beginning of the string.")
```

Q2. Accept filename and byte position. If file exists, move pointer to that position and print content from there to end.

```
import os

fname = input("Enter filename: ")
if os.path.exists(fname):
    pos = int(input("Enter byte position: "))
    with open(fname, "r") as f:
        f.seek(pos)
        print("Content from position", pos, ":\n", f.read())
else:
    print("File does not exist.")
```

Q2 (OR). Create module variables.py with factorial, power dict, vowels list. Import and use.

```
# File: variables.py
import math
def factorial(n):
    return math.factorial(n)
power = {1:1, 2:4, 3:9, 4:16, 5:25}
vowels = ['a', 'e', 'i', 'o', 'u']

# File: main.py
import variables
n = int(input("Enter n: "))
print("Factorial:", variables.factorial(n))
print("Power:", variables.power.get(n, n**2))
print("Second vowel:", variables.vowels[1])
```

Slip 15

Q1. Get current time. a) Convert seconds to struct_time in UTC b) Convert struct_time to formatted string.

```
import time

t = time.time()
utc = time.gmtime(t)
print("struct_time (UTC):", utc)
formatted = time.strftime("%A, %d %B %Y %H:%M:%S", utc)
print("Formatted string:", formatted)
```

Q2. Flask app: Library Book List with table and quote.

```
# File: app.py
from flask import Flask
app = Flask(__name__)

@app.route('/')
def library():
    return '''
<html><body>
<h1>City Public Library</h1>
<table border="1">
  <tr><th>Title</th><th>Author</th></tr>
  <tr><td>Python Crash Course</td><td>Eric Matthes</td></tr>
  <tr><td>Clean Code</td><td>Robert Martin</td></tr>
  <tr><td>The Alchemist</td><td>Paulo Coelho</td></tr>
</table>
<p><b style="color:blue">"A reader lives a thousand lives before he dies."</b></p>
</body></html>'''

if __name__ == '__main__':
    app.run(debug=True)
```

Q2 (OR). Check whether a given URL is valid using RE.

```
import re

url = input("Enter URL: ")
pattern = r'^(https?://)?(www\.)?[a-zA-Z0-9\-]+\.[a-zA-Z]{2,}(/.*)?$'
if re.match(pattern, url):
    print("Valid URL.")
else:
    print("Invalid URL.")
```

Slip 16

Q1. Use regex to validate three-letter word patterns like bit, but, bat, hit, hut, hat.

```
import re

word = input("Enter word: ")
pattern = r'^[bh][aiu]t$'
if re.fullmatch(pattern, word):
    print(f"'{word}' matches the pattern.")
else:
    print(f"'{word}' does NOT match the pattern.")
```

Q2. Create IPL DataFrame. a) First 2 rows b) Last 3 rows c) Add null values d) Most expensive player e) Players per team.

```
import pandas as pd
import numpy as np

data = {
    "Player": ["Hardik Pandya","K L Rahul","Andre Russel","Jasprit Bumrah","Virat Kohli","Rohit
Sharma"],
    "Team": ["Mumbai Indians","Kings Eleven","Kolkata Knight Riders","Mumbai Indians","RCB","Mumbai
Indians"],
    "Category": ["Batsman","Batsman","Batsman","Bowler","Batsman","Batsman"],
    "BidPrice": [13, 12, 7, 10, 17, 15],
    "Runs": [1000, 2400, 900, 200, 3600, 3700]
}
df = pd.DataFrame(data)

print("a) First 2 rows:\n", df.head(2))
print("\nb) Last 3 rows:\n", df.tail(3))
df.loc[0, "BidPrice"] = np.nan
df.loc[2, "Runs"] = np.nan
print("\nc) After adding nulls:\n", df)
print("\nd) Most expensive:", df.loc[df["BidPrice"].idxmax(), "Player"])
print("\ne) Players per team:\n", df["Team"].value_counts())
```

Q2 (OR). Create package vehicle with sub-package types containing cars.py and bikes.py.

```
# vehicle/__init__.py  → (empty)
# vehicle/types/__init__.py  → (empty)

# vehicle/types/cars.py
def available_models():
    return ["Swift", "Innova", "Creta", "Nexon"]

# vehicle/types/bikes.py
def available_models():
    return ["Splendor", "Pulsar", "Activa", "Duke"]

# main.py
from vehicle.types import cars, bikes
print("Cars:", cars.available_models())
print("Bikes:", bikes.available_models())
```

Slip 17

Q1. Accept string. Replace all spaces with colon and all commas with dot. Display modified string.

```
s = input("Enter string: ")
s = s.replace(" ", ":").replace(",", ".")
print("Modified string:", s)
```

Q2. Create list, string and tuple. Use random module to display unique multiple random elements.

```
import random

lst = [10, 20, 30, 40, 50, 60]
s = "Hello World Python"
tup = (1, 2, 3, 4, 5, 6, 7, 8)

print("Random from list:", random.sample(lst, 3))
print("Random from string:", random.sample(s, 4))
print("Random from tuple:", random.sample(tup, 3))
```

Q2 (OR). Accept number. a) Armstrong check b) Sum of digits till single digit.

```
def is_armstrong(n):
    digits = str(n)
    return n == sum(int(d)**len(digits) for d in digits)

def digit_sum(n):
    while n >= 10:
        n = sum(int(d) for d in str(n))
    return n

n = int(input("Enter number: "))
print("Armstrong:", is_armstrong(n))
print("Digit sum till single:", digit_sum(n))
```

Slip 18

Q1. List all files and folders in current directory using os.listdir().

```
import os

entries = os.listdir(".")
print("Files and Folders in current directory:")
for entry in entries:
    print(entry)
```

Q2. Validate mobile number using regex (10 digits starting 6-9, or with 0 prefix = 11 digits, or 91 prefix = 12 digits).

```
import re

mobile = input("Enter mobile number: ")
pattern = r'^(0|91)?[6-9][0-9]{9}$'
if re.fullmatch(pattern, mobile):
    print("Valid mobile number.")
else:
    print("Invalid mobile number.")
```

Q2 (OR). Build package **education** with sub-package **courses** containing science.py and arts.py with get_subjects().

```
# education/__init__.py  → (empty)
# education/courses/__init__.py  → (empty)

# education/courses/science.py
def get_subjects():
    return ["Physics", "Chemistry", "Biology", "Maths"]

# education/courses/arts.py
def get_subjects():
    return ["History", "Geography", "Economics", "Sociology"]

# main.py
from education.courses import science, arts
print("Science subjects:", science.get_subjects())
print("Arts subjects:", arts.get_subjects())
```

Slip 19

Q1. Shuffle a deck of cards. Display cards obtained in 5 attempts.

```
import random

suits = ["Hearts", "Diamonds", "Clubs", "Spades"]
ranks = ["A","2","3","4","5","6","7","8","9","10","J","Q","K"]
deck = [r + " of " + s for s in suits for r in ranks]

random.shuffle(deck)
print("Cards in 5 attempts:")
for i in range(5):
    print(f"Attempt {i+1}: {deck[i]}")
```

Q2. Display all entries in current directory. Identify each as file or folder using getcwd().

```
import os

path = os.getcwd()
print("Current directory:", path)
for entry in os.listdir(path):
    full = os.path.join(path, entry)
    if os.path.isfile(full):
        print(entry, "- File")
    else:
        print(entry, "- Folder")
```

Q2 (OR). Create IPL DataFrame with nulls. a) First 2 rows b) Last 3 rows c) Check nulls d) Replace with mean e) Most expensive f) Players per team.

```
import pandas as pd
import numpy as np

data = {
    "Player": ["Hardik Pandya","K L Rahul","Andre Russel","Jasprit Bumrah","Virat Kohli","Rohit
Sharma"],
    "Team": ["Mumbai Indians","Kings Eleven","Kolkata Knight Riders","Mumbai Indians","RCB","Mumbai
Indians"],
    "Category": ["Batsman","Batsman","Batsman","Bowler","Batsman","Batsman"],
    "BidPrice": [np.nan, 12, 7, 10, np.nan, 15],
    "Runs": [1000, 2400, 900, np.nan, 3600, 3700]
}
df = pd.DataFrame(data)
print("a) First 2:\n", df.head(2))
print("\nb) Last 3:\n", df.tail(3))
print("\nc) Null values:\n", df.isnull().sum())
df["BidPrice"].fillna(df["BidPrice"].mean(), inplace=True)
df["Runs"].fillna(df["Runs"].mean(), inplace=True)
print("\nd) After filling nulls:\n", df)
print("\ne) Most expensive:", df.loc[df["BidPrice"].idxmax(), "Player"])
print("\nf) Players per team:\n", df["Team"].value_counts())
```

Slip 20

Q1. Accept a string. Count vowels, consonants and special symbols.

```
s = input("Enter string: ")
vowels = consonants = special = 0
for ch in s:
    if ch.lower() in "aeiou":
        vowels += 1
    elif ch.isalpha():
        consonants += 1
    elif not ch.isspace():
        special += 1
print("Vowels:", vowels)
print("Consonants:", consonants)
print("Special:", special)
```

Q2. Accept filename and byte position. Seek to position and display content from there to end.

```
import os

fname = input("Enter filename: ")
if os.path.exists(fname):
    pos = int(input("Enter byte position: "))
    with open(fname, "r") as f:
        f.seek(pos)
        print("Content from position", pos, ":\n", f.read())
else:
    print("File does not exist.")
```

Q2 (OR). Create student DataFrame. a) GroupBy school b) Type of GroupBy object c) Mean/min/max of age per group d) Students with weight > 32.

```
import pandas as pd

data = {
    "school": ["S1","S2","S3","S4","S5","S6"],
    "class": ["S001","S002","S003","S004","S005","S006"],
    "name": ["Asha","Fransis","Charlie","David","Shyam","Eashel"],
    "age": [12, 12, 13, 14, 12, 16],
    "height": [173, 192, 186, 167, 151, 159],
    "weight": [35, 32, 33, 30, 31, 32]
}
df = pd.DataFrame(data)

grp = df.groupby("school")
print("a) GroupBy created.")
print("b) Type:", type(grp))
print("c) Age stats:\n", grp["age"].agg(["mean","min","max"]))
print("d) Students with weight > 32:", len(df[df["weight"] > 32]))
```

Slip 21

Q1. Create DataFrame from {Subject_Name: Marks} dictionary. Display first 5 and last 5 rows.

```
import pandas as pd

data = {
    "Subject_Name": ["Maths","Physics","Chemistry","Biology","English",
                     "History","Geography","CS","Economics","PE"],
    "Marks": [85, 78, 90, 72, 88, 65, 70, 95, 60, 80]
}
df = pd.DataFrame(data)
print("First 5 rows:\n", df.head())
print("\nLast 5 rows:\n", df.tail())
```

Q2. Accept filename. If exists, display contents with count of characters, words, lines.

```
import os

fname = input("Enter filename: ")
if os.path.exists(fname):
    with open(fname, "r") as f:
        content = f.read()
    print("Contents:\n", content)
    print("Characters:", len(content))
    print("Words:", len(content.split()))
    print("Lines:", content.count('\n') + 1)
else:
    print("File does not exist.")
```

Q2 (OR). Package banking with account.py and loan.py.

```
# banking/__init__.py  → (empty)

# banking/account.py
def open_account(name):
    print(f"Account created for {name}.")

# banking/loan.py
def apply_loan(amount):
    print(f"Loan of Rs.{amount} applied successfully.")

# main.py
from banking import account, loan
name = input("Name: ")
account.open_account(name)
amount = float(input("Loan amount: "))
loan.apply_loan(amount)
```

Slip 22

Q1. Accept m and n. Divide m by n. Handle ZeroDivisionError.

```
try:
    m = float(input("Enter m: "))
    n = float(input("Enter n: "))
    print("Result:", m / n)
except ZeroDivisionError:
    print("Error: Cannot divide by zero.")
```

Q2. Create Student1 and Student2 DataFrames. Merge into third DataFrame.

```
import pandas as pd

s1 = pd.DataFrame({
    "ID": ["S1","S2","S3","S4","S5"],
    "Name": ["Anita","Sakshi","Om","Raj","Kanika"],
    "Age": [18, 23, 20, 21, 20]
})
s2 = pd.DataFrame({
    "ID": ["S2","S4","S5","S6","S7"],
    "City": ["Pune","Nashik","Mumbai","Delhi","Nanded"],
    "Marks": [97, 78, 86, 66, 54]
})
s3 = pd.merge(s1, s2, on="ID", how="outer")
print("Student1:\n", s1)
print("\nStudent2:\n", s2)
print("\nMerged:\n", s3)
```

Q2 (OR). Class iterator generating factorial series up to n.

```
class FactorialSeries:
    def __init__(self, n):
        self.n = n
        self.current = 1
        self.fact = 1

    def __iter__(self):
        return self

    def __next__(self):
        if self.current > self.n:
            raise StopIteration
        self.fact *= self.current
        self.current += 1
        return self.fact

n = int(input("Enter n: "))
for val in FactorialSeries(n):
    print(val, end=" ")
```

Slip 23

Q1. Define productInfo(name, price=100). Display product name only if price < 100.

```
def productInfo(name, price=100):
    if price < 100:
        print("Product:", name)
    else:
        print(f"Price Rs.{price} is not less than Rs.100. Not displayed.")

productInfo("Pen", 50)
productInfo("Notebook")
productInfo("Book", 120)
```

Q2. a) Local and UTC time b) ctime() and gmtime() c) Accept datetime string d) Parse with strptime() and format with strftime() in two formats.

```
import time

local = time.localtime()
utc = time.gmtime()
print("Local time (ctime):", time.ctime())
print("UTC time:", time.strftime("%Y-%m-%d %H:%M:%S", utc))

dt_str = input("Enter datetime (DD-MM-YYYY HH:MM:SS): ")
t = time.strptime(dt_str, "%d-%m-%Y %H:%M:%S")
print("Format 1:", time.strftime("%A, %d %B %Y", t))
print("Format 2:", time.strftime("%d/%m/%Y %I:%M %p", t))
```

Q2 (OR). Accept string. a) Split into words b) Words without vowels c) Sort alphabetically d) Count.

```
import re

s = input("Enter string: ")
words = s.split()
no_vowel = sorted([w for w in words if not re.search(r'[aeiou]', w, re.IGNORECASE)])
print("Words without vowels (sorted):", no_vowel)
print("Count:", len(no_vowel))
```

Slip 24

Q1. Extract numbers from a given string using RE.

```
import re

s = input("Enter string: ")
numbers = re.findall(r'\d+', s)
print("Extracted numbers:", numbers)
```

Q2. a) Define check_pass_fail(marks) returning Pass/Fail b) Accept n students and their marks c) Display result d) Count passed and failed.

```
def check_pass_fail(marks):
    return "Pass" if marks >= 40 else "Fail"

n = int(input("Enter number of students: "))
passed = failed = 0
for i in range(1, n+1):
    marks = int(input(f"Marks of student {i}: "))
    result = check_pass_fail(marks)
    print(f"Student {i}: {result}")
    if result == "Pass":
        passed += 1
    else:
        failed += 1

print("Total Passed:", passed)
print("Total Failed:", failed)
```

Q2 (OR). Flask app: College Page with name, courses, image.

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def college():
    return '''
<html><body>
<h1 style="text-align:center">SPPU College of Science</h1>
<h3>Courses:</h3>
<ul>
  <li>B.Sc. Computer Science</li>
  <li>B.Sc. Mathematics</li>
  <li>B.Sc. Physics</li>
</ul>
<img src="https://via.placeholder.com/400x200?text=College" alt="College">
</body></html>'''

if __name__ == '__main__':
    app.run(debug=True)
```

Slip 25

Q1. Shuffle a deck of cards. Display cards in 5 attempts.

```
import random

suits = ["Hearts", "Diamonds", "Clubs", "Spades"]
ranks = ["A","2","3","4","5","6","7","8","9","10","J","Q","K"]
deck = [r + " of " + s for s in suits for r in ranks]

random.shuffle(deck)
for i in range(5):
    print(f"Attempt {i+1}: {deck[i]}")
```

Q2. Package finance_tools with interest.py (simple interest at 5%) and currency.py (INR to USD at 0.012).

```
# finance_tools/__init__.py  → (empty)

# finance_tools/interest.py
def cal_interest(amount, year):
    return (amount * 5 * year) / 100

# finance_tools/currency.py
def convert(amount):
    return amount * 0.012

# main.py
from finance_tools import interest, currency
amount = float(input("Amount: "))
year = int(input("Years: "))
print("SI:", interest.cal_interest(amount, year))
print("USD:", currency.convert(amount))
```

Q2 (OR). Check whether a given URL is valid using RE.

```
import re

url = input("Enter URL: ")
pattern = r'^(https?://)?(www\.)?[a-zA-Z0-9\-]+\.[a-zA-Z]{2,}(/.*)?$'
if re.match(pattern, url):
    print("Valid URL.")
else:
    print("Invalid URL.")
```

Slip 26

Q1. Remove all whitespaces from a given string. Display modified string.

```
s = input("Enter string: ")
s = s.replace(" ", "")
print("Modified string:", s)
```

Q2. Package vehicle with sub-package types containing cars.py and bikes.py.

```
# vehicle/__init__.py  → (empty)
# vehicle/types/__init__.py  → (empty)

# vehicle/types/cars.py
def available_models():
    return ["Swift", "Innova", "Creta", "Nexon"]

# vehicle/types/bikes.py
def available_models():
    return ["Splendor", "Pulsar", "Activa", "Duke"]

# main.py
from vehicle.types import cars, bikes
print("Cars:", cars.available_models())
print("Bikes:", bikes.available_models())
```

Q2 (OR). Stationery DataFrame. a) Pencil rows b) Eraser count=25 c) Company & Price d) Pencil & Pen rows e) Add Colour column.

```
import pandas as pd

data = {
    "Company": ["Apsara","Natraj","Cello","Parkar","Apsara"],
    "Count": [15, 20, 25, 35, 20],
    "Price": [250, 200, 600, 900, 300]
}
index = ["Pencil","Pencil","Pen","Pen","Eraser"]
df = pd.DataFrame(data, index=index)

print("a) Pencil:\n", df.loc["Pencil"])
df.loc["Eraser","Count"] = 25
print("\nb) After change:\n", df)
print("\nc) Company & Price:\n", df[["Company","Price"]])
print("\nd) Pencil & Pen:\n", df.loc[["Pencil","Pen"]])
df["Colour"] = ["Blue","Red","Black","Gold","Pink"]
print("\ne) With Colour column:\n", df)
```

Slip 27

Q1. Generate random radius. Compute circumference of circle using math module (π = 3.14).

```
import random
import math

r = random.randint(1, 100)
circumference = 2 * 3.14 * r
print("Random radius:", r)
print("Circumference:", circumference)
```

Q2. Validate password: at least 1 lowercase, 1 digit, 1 special char ($,#,@), length 6–12.

```
import re

password = input("Enter password: ")
if len(password) < 6 or len(password) > 12:
    print("Invalid: Length must be 6–12.")
elif not re.search(r'[a-z]', password):
    print("Invalid: Must contain at least one lowercase letter.")
elif not re.search(r'[0-9]', password):
    print("Invalid: Must contain at least one digit.")
elif not re.search(r'[$#@]', password):
    print("Invalid: Must contain at least one of $, #, @.")
else:
    print("Valid password.")
```

Q2 (OR). List all files in current directory. Display .py count and content of one .py file.

```
import os

files = os.listdir(".")
py_files = [f for f in files if f.endswith(".py")]
print("All files:", files)
print("Total .py files:", len(py_files))

if py_files:
    with open(py_files[0], "r") as f:
        print(f"\nContents of {py_files[0]}:\n", f.read())
```

Slip 28

Q1. Use regex to validate three-letter word patterns like cat, cot, cut, mat, met, mud.

```
import re

word = input("Enter word: ")
pattern = r'^[cm][aou][td]$'
if re.fullmatch(pattern, word):
    print(f"'{word}' matches the pattern.")
else:
    print(f"'{word}' does NOT match the pattern.")
```

Q2. Package education with sub-package courses containing science.py and arts.py.

```
# education/__init__.py  → (empty)
# education/courses/__init__.py  → (empty)

# education/courses/science.py
def get_subjects():
    return ["Physics", "Chemistry", "Biology", "Maths"]

# education/courses/arts.py
def get_subjects():
    return ["History", "Geography", "Economics", "Sociology"]

# main.py
from education.courses import science, arts
print("Science:", science.get_subjects())
print("Arts:", arts.get_subjects())
```

Q2 (OR). Display all entries in current directory. Identify each as file or folder using getcwd().

```
import os

path = os.getcwd()
print("Current directory:", path)
for entry in os.listdir(path):
    full = os.path.join(path, entry)
    kind = "File" if os.path.isfile(full) else "Folder"
    print(entry, "-", kind)
```

Slip 29

Q1. Create DataFrame using Iris.csv. Display contents and statistical info.

```
import pandas as pd

# If Iris.csv not available, create sample data
try:
    df = pd.read_csv("Iris.csv")
except FileNotFoundError:
    data = {
        "sepal_length": [5.1, 4.9, 4.7, 4.6, 5.0],
        "sepal_width": [3.5, 3.0, 3.2, 3.1, 3.6],
        "petal_length": [1.4, 1.4, 1.3, 1.5, 1.4],
        "petal_width": [0.2, 0.2, 0.2, 0.2, 0.2],
        "species": ["setosa"]*5
    }
    df = pd.DataFrame(data)

print(df)
print("\nStatistical Info:\n", df.describe())
```

Q2. Accept text filename and a string. If file exists, append string to file and display contents.

```
import os

fname = input("Enter filename: ")
if os.path.exists(fname):
    text = input("Enter string to append: ")
    with open(fname, "a") as f:
        f.write("\n" + text)
    with open(fname, "r") as f:
        print("File contents:\n", f.read())
else:
    print("File does not exist.")
```

Q2 (OR). Validate mobile number using regex (10, 11, or 12 digits).

```
import re

mobile = input("Enter mobile number: ")
pattern = r'^(0|91)?[6-9][0-9]{9}$'
if re.fullmatch(pattern, mobile):
    print("Valid mobile number.")
else:
    print("Invalid mobile number.")
```

Slip 30

Q1. Replace all occurrences of 'a' with '*' in a string.

```
s = input("Enter string: ")
result = s.replace('a', '*').replace('A', '*')
print("Modified string:", result)
```

Q2. a) Define check_pass_fail() b) Accept n students and marks c) Display results d) Count passed and failed.

```
def check_pass_fail(marks):
    return "Pass" if marks >= 40 else "Fail"

n = int(input("Enter number of students: "))
passed = failed = 0
for i in range(1, n+1):
    marks = int(input(f"Marks of student {i}: "))
    result = check_pass_fail(marks)
    print(f"Student {i}: {result}")
    if result == "Pass":
        passed += 1
    else:
        failed += 1

print("Total Passed:", passed)
print("Total Failed:", failed)
```

Q2 (OR). a) Local and UTC time b) ctime() and gmtime() c) Accept datetime string d) Parse and format in two ways.

```
import time

print("Local time (ctime):", time.ctime())
print("UTC time:", time.strftime("%Y-%m-%d %H:%M:%S", time.gmtime()))

dt_str = input("Enter datetime (DD-MM-YYYY HH:MM:SS): ")
t = time.strptime(dt_str, "%d-%m-%Y %H:%M:%S")
print("Format 1:", time.strftime("%A, %d %B %Y", t))
print("Format 2:", time.strftime("%d/%m/%Y %I:%M %p", t))
```